システム連携の基礎

REST API を使ったデータ取得と業務改善

── ブラウザと Python ではじめる外部データ活用 ──

河合 勝彦
名古屋市立大学大学院経済学研究科
kkawai@econ.nagoya-cu.ac.jp

2026 年 10 月 9 日

目次

はじめに（環境準備） 5

第 I 部　概念編：REST API とは何か 9

第 1 章　API と REST 10
1.1　身近にある API . . . 10
1.2　最低限おさえる用語 . . . 10
1.3　REST API の考え方 . . . 10
1.4　REST を読むための 2 つの視点 . . . 11
1.5　API は画面ではなく契約である . . . 11

第 2 章　HTTP の基本 13
2.1　URL の構成 . . . 13
2.2　HTTP メソッド . . . 13
2.3　リクエストとレスポンスの構造 . . . 14
2.4　HTTP メッセージの実物 . . . 15
2.5　ステータスコード . . . 15
2.6　API でよく見るステータスコード . . . 16

第 3 章　JSON の読み方 17
3.1　JSON の構造 . . . 17
3.2　入れ子と配列を読む . . . 17
3.3　JSON と Python の対応 . . . 18

第 II 部　体験編：ブラウザで API を試す 20

第 4 章　アドレスバーで API を呼ぶ 22
4.1　JSONPlaceholder を開いてみる . . . 22
4.2　URL の末尾を変えてみる . . . 22

第 5 章　開発者ツール（DevTools）で観察する 24
5.1　Network タブで観察する . . . 24

5.2 Console で fetch() を使う 24

第 III 部　実装編：Python で API を呼び出す 27

第 6 章　最初のリクエスト 29
6.1 GET して，ステータスと JSON を見る 29
6.2 API 呼び出しの基本形 30
6.3 JSON から必要な値を取り出す 30
6.4 この章の要点 31

第 7 章　パラメータとエラー処理 32
7.1 クエリパラメータで絞り込む 32
7.2 エラー処理 33
7.3 ステータスコードから原因を推測する 34

第 IV 部　応用編：実データ API を業務に活かす 36

第 8 章　天気データを取得する（Open-Meteo） 38
8.1 現在の気温・風速を取得する 38
8.2 パラメータの意味 39
8.3 Open-Meteo レスポンスの読み方 39
8.4 時間別予報を表にする 40
8.5 グラフにする 41
8.6 業務判断に結びつける 43

第 9 章　（発展）GitHub REST API 45
9.1 公開リポジトリの情報を取得する 45
9.2 ヘッダーもデータである 46

第 10 章　CRUD を体験する（疑似 API） 48
10.1 CRUD と HTTP メソッドの対応 48
10.2 Python で CRUD を送る 48
10.3 作成・更新・削除で確認すること 50

第 V 部　運用編：安全な利用と業務システムへの展開 51

第 11 章　API 利用上の注意 52
11.1 認証・API キー 52
11.2 レート制限・利用規約 53
11.3 個人情報・データ品質 53
11.4 ページング・キャッシュ・リトライ 53

第 12 章　業務システムでの活用（MIS の視点）　56
12.1　API 連携を設計する手順 56

第 VI 部　発展編：コードを整理し，仕様を読む　58

第 13 章　Python ライブラリで API 利用を楽にする　60
13.1　代表的なライブラリの使い分け 60
13.2　requests.Session で共通設定をまとめる 61
13.3　httpx.Client でベース URL を隠す 61
13.4　小さな API クライアントを書く 62
13.5　SDK と OpenAPI 生成クライアント 63

第 14 章　OpenAPI で API 仕様を読む　64
14.1　OpenAPI 仕様書に書かれること 64
14.2　OpenAPI を使う利点と注意点 65

付録 A　ワークシート　66

付録 B　ミニ課題　68

付録 C　評価ルーブリック　69

付録 D　API 利用チェックリスト　70

付録 E　よくあるトラブルと切り分け　71

付録 F　授業計画（シラバス例）　72
F.1　90 分 × 1 回（1 回完結）. 72
F.2　90 分 × 3 回 72
F.3　90 分 × 5 回（ミニプロジェクト型）. 72

付録 G　初心者向け公開 API 集　73

付録 H　スクリプト一覧と参考リンク　75
H.1　スクリプト一覧 75
H.2　参考リンク 75

はじめに（環境準備）

本書のねらい

本書は，経営情報システム（MIS）を学ぶ学生のための REST API 実習教材です。ねらいは「REST とは何か」を理論として暗記することではありません。**API ドキュメントを読み，リクエストを送り，返ってきた JSON から必要な値を取り出し，業務で使える形に加工する** ── この一連の流れを，実際に手を動かして体験することを目標とします。

そのため本書は，技術そのものより **システム間連携・外部データ活用・業務アプリとの接続**に重点を置いて構成しています。天気データを取得して「小売や飲食の業務判断にどう使うか」を考えるところまで進みます。

到達目標

本書を終えると，次の 5 つができるようになります。

到達目標	学生ができること
REST API の基本を説明できる	エンドポイント・HTTP メソッド・ステータスコード・JSON を説明できる
API ドキュメントを読める	URL・パラメータ・認証・レスポンス例を読み取れる
実際に API を呼び出せる	ブラウザや Python から GET リクエストを送れる
レスポンスを処理できる	JSON から必要な値を取り出し，表やグラフにできる
業務利用を考えられる	API 連携が企業システムでどう使われるか説明できる

本書の読み方

本書は，短時間の授業でも，複数回の実習でも使えるように，次の順序で構成しています。最初からすべての技術を覚えようとせず，「**URL を開く**」「**レスポンスを見る**」「**必要な値を取り出す**」「**業務判断に使う**」という流れを意識してください。

部	位置づけ	学ぶこと
概念編	API の見取り図	API，REST，HTTP，JSON の基本用語を整理する
体験編	ブラウザで観察	URL を開き，DevTools で実際のリクエストとレスポンスを見る
実装編	Python で取得	`requests` を使い，JSON から必要な値を取り出す
応用編	実データ活用	天気や公開リポジトリのデータを業務判断に結びつけ，CRUD 操作も体験する
運用編	実務への橋渡し	認証，レート制限，ページング，リトライ，データ品質と業務システム連携を学ぶ
発展編	コードと仕様の整理	ライブラリ・SDK によるコード整理と OpenAPI 仕様書の読み方を知る

学習の型

API 利用は毎回，おおよそ同じ手順で進みます。**ドキュメントを読む → リクエストを送る → ステータスコードを見る → JSON を読む →必要な値を取り出す → 業務での使い道を考える**，という型です。この型を身につけると，新しい API に出会っても落ち着いて読み解けます。

必要なものは「ブラウザ」と「Python」だけ

本書では，特別な有料ツールや API キーは使いません。用意するのは次の 2 つだけです。

- **Web ブラウザ**（Chrome・Edge・Firefox など。開発者ツールが使えるもの）
- **Python の実行環境**（本書では uv を使います）

uv のインストール

uv は高速な Python 実行・パッケージ管理ツールです。次のコマンドで導入できます。

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

スクリプトの動かし方（PEP 723）

本書の Python スクリプトは **PEP 723**（インラインスクリプトメタデータ）という形式で書かれています。ファイルの先頭に，必要なライブラリを次のように宣言しておきます。

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
import requests
...
```

この宣言があるおかげで，仮想環境を自分で作らなくても，次の 1 行で実行できます。uv が必要なライブラリ（ここでは `requests`）を自動でそろえてから実行します。

```
uv run scripts/01_first_request.py
```

ここがポイント

ライブラリのインストールやバージョン管理を意識せずに済むのが PEP 723 ＋ uv の利点です。配布されたスクリプトを「そのまま」実行できるので，環境構築でつまずきません。

本書で使う API

最初に複雑な認証でつまずかないよう，原則として **API キーが不要な API** を使います。

API	用途
JSONPlaceholder	練習用の疑似 REST API。安全に GET/POST/PUT/DELETE を試せる
Open-Meteo	天気予報 API。API キー不要で実データを取得できる
GitHub REST API	（発展）公開リポジトリの情報取得。認証なしでも利用可能

API ドキュメントで最初に見るところ

API を使うときは，コードを書く前にドキュメントから次の点を読み取ります。

確認項目	読み取る内容
エンドポイント	どの URL にアクセスするか
HTTP メソッド	GET か，POST/PUT/DELETE か
パラメータ	必須項目・任意項目・単位・既定値
認証	API キーやトークンが必要か
レスポンス	どのキーに必要な値が入っているか
制限	レート制限，利用規約，商用利用の可否

第 I 部

概念編：REST API とは何か

第 1 章

API と REST

1.1　身近にある API

スマートフォンやパソコンで使うサービスの多くは，裏側で別のシステムからデータや機能を「借りて」動いています。例えば次のような場面です。

- スマホアプリで**天気**を表示する
- EC サイトで**配送状況**を確認する
- 大学ポータルが**外部認証サービス**と連携する
- 会計システムが**銀行明細**を取得する

これらはいずれも「別のシステムからデータや機能を借りてくる」仕組みであり，その代表的な方法が **API**（Application Programming Interface）です。

1.2　最低限おさえる用語

本書で使う用語はこれだけで十分です。

用語	説明
API	アプリケーション同士がやり取りするための窓口
REST API	HTTP を使ってリソースを操作する API の設計スタイル
エンドポイント	API の URL
リクエスト	利用者側から API に送る要求
レスポンス	API から返ってくる結果
JSON	API でよく使われるデータ形式
HTTP メソッド	GET・POST・PUT・DELETE などの操作種別
ステータスコード	成功・失敗を示す番号

1.3　REST API の考え方

REST API は，**URL（エンドポイント）でリソースを指定し，HTTP メソッドで操作する**という考え方に基づきます。たとえば「投稿（post)」というリソースに対して，

- `GET /posts/1` なら「1 番の投稿を取得する」
- `POST /posts` なら「新しい投稿を作る」
- `DELETE /posts/1` なら「1 番の投稿を削除する」

のように，「**何を（URL）**」「**どうする（メソッド）**」の組み合わせで操作を表します。

1.4 REST を読むための 2 つの視点

REST は製品名でも特定のプログラミング言語でもなく，Web の仕組みを使って API を設計するための考え方です。初学者は，まず次の 2 点を押さえれば十分です。

視点	説明
リソース指向	API が扱う対象を「投稿」「ユーザー」「注文」「在庫」のような名詞で考える。URL はその対象を指す名前になる。
ステートレス	1 回のリクエストに，処理に必要な情報をできるだけ含める。サーバー側の前回状態に強く依存しないようにする。

たとえば `/orders/123` は「123 番の注文」というリソースを表します。`GET /orders/123` なら取得，`DELETE /orders/123` なら削除，というように同じ URL でもメソッドが変わると操作が変わります。反対に，`/getOrder?id=123` や `/deleteOrder?id=123` のように動詞を URL に入れる設計も実在しますが，REST の学習では「URL は名詞，メソッドは動詞」と分けて考えると整理しやすくなります。
ステートレスの例として，認証が必要な API では「前回ログインしたから今回は省略」とはせず，リクエストごとにトークンをヘッダーに付けて送ります。各リクエストが独立しているので，サーバーを増やして負荷を分散しやすく，失敗したリクエストだけを送り直すことも容易になります。

1.5 API は画面ではなく契約である

Web ページは人間が読む画面ですが，API はプログラムが読む約束事です。そのため，API を使うときは画面の見た目ではなく，次のような**契約**を確認します。

- どの URL に，どのメソッドで送るか
- 必須パラメータと任意パラメータは何か
- 成功時・失敗時に，どのステータスコードと JSON が返るか
- 認証，レート制限，利用規約はどうなっているか

この契約を読めるようになることが，API ドキュメントを読めるということです。

章末練習：解答例つき

1. スマホの天気アプリで，リクエストを送る側とレスポンスを返す側はそれぞれ何か。
   解答例：アプリがクライアントとして天気 API にリクエストを送り，天気 API のサーバーが JSON などのレスポンスを返す。

2. `GET /posts/1` は，何をどうする操作か。
   解答例：`/posts/1` で「1 番の投稿」を指定し，`GET` で「取得する」操作を表す。
3. REST API で URL と HTTP メソッドを分けて考える理由を説明しなさい。
   解答例：URL が対象リソースを示し，HTTP メソッドが操作種別を示すため，API の意味を一貫して読み取りやすくなる。

第 2 章

HTTP の基本

2.1　URL の構成

API の URL は, いくつかの部品に分けて読むと意味が分かりやすくなります。本書の Open-Meteo の例を分解してみましょう。

```
https://api.open-meteo.com/v1/forecast?latitude=35.1815&longitude=136.9066
```

部品	この例での値	役割
スキーム	`https`	通信方式。API では暗号化された HTTPS を使う
ホスト	`api.open-meteo.com`	API を提供しているサーバー
パス	`/v1/forecast`	どのリソース（機能）を使うか。`v1` は API のバージョン
クエリ文字列	`?latitude=...`	絞り込みや条件指定。`?` の後に **キー=値** を書き，`&` で複数つなぐ

値を URL に埋め込む方法は 2 通りあります。`/posts/1` の `1` のように**パスの一部で対象を特定する**もの（パスパラメータ）と，`?postId=1` のように**条件を付け加える**もの（クエリパラメータ）です。「どれか 1 件を指す」ならパス,「一覧を絞り込む・並べ替える」ならクエリ，と考えると整理できます。

2.2　HTTP メソッド

リクエストの「操作の種類」を表すのが HTTP メソッドです。標準的な説明は MDN Web Docs にまとめられています（巻末リンク参照)。

メソッド	役割	本書での例
GET	リソースの取得	投稿や天気データを取得する
POST	データの送信（作成）	新しい投稿を作成する
PUT	リソースの作成・置換	既存の投稿を置き換える
PATCH	リソースの部分更新	一部の項目だけ修正する
DELETE	リソースの削除	投稿を削除する

2.2.1　安全性とべき等性

HTTP メソッドは，単に名前が違うだけではありません。実務では，その操作が**安全**か，**べき等**かを意識します。ここでいう安全とは「サーバー上の業務データを変更しない」という意味です。べき等とは「同じリクエストを 1 回送っても複数回送っても，最終的な状態が同じ」という意味です。

メソッド	安全か	べき等か	実務上の見方
GET	はい	はい	取得だけなので，再読み込みしても通常はデータを変えない
POST	いいえ	通常いいえ	作成や処理実行に使われ，同じ送信で重複作成が起きることがある
PUT	いいえ	はい	同じ内容で何度置き換えても，最終的な状態は同じになる
PATCH	いいえ	保証されない	差分更新なので，「在庫を 1 減らす」のような更新は回数分だけ結果が変わる
DELETE	いいえ	はい	1 回目で削除され，2 回目以降も「存在しない」状態が続く

べき等性は「サーバーの最終的な状態」についての性質であり，返ってくるステータスコードが毎回同じという意味ではありません。たとえば DELETE を 2 回送ると，1 回目は 200 や 204，2 回目は 404 が返ることがありますが，「削除されている」という状態は変わりません。なお，HTTP の仕様（RFC 9110）が定めるのはあくまで約束事であり，実際の API がそれに従っているかはドキュメントで確認します。

メソッドの性質

GET は原則として読み取り専用です。ブラウザで URL を開くだけなら，通常はデータを変更しません。一方，POST・PUT・DELETE はデータを変える可能性があります。実務では，本番環境に送る前に「どのデータが変わるか」を必ず確認します。

2.3　リクエストとレスポンスの構造

リクエスト（要求）とレスポンス（結果）は，それぞれ次の要素から成り立っています。

リクエストの要素	内容
URL	どのリソースを対象にするか
メソッド	GET・POST など，何をするか
ヘッダー	付加情報（データ形式や認証情報など）
パラメータ	絞り込み条件など（`?key=value`）
ボディ	送信するデータ本体（POST/PUT のとき）

レスポンスの要素	内容
ステータスコード	成功・失敗を示す番号（例：200, 404）
ヘッダー	付加情報（データ形式・レート制限など）
ボディ	返ってくるデータ本体（多くは JSON）

2.4 HTTP メッセージの実物

リクエストとレスポンスは, 実際にはテキスト形式のメッセージとしてやり取りされます(HTTP/2 以降は内部的にはバイナリですが，意味は同じです)。`GET /posts/1` を送ったときのやり取りを，主要な部分だけ抜き出すと次のようになります。

```
GET /posts/1 HTTP/1.1                    ← リクエスト行（メソッド・パス）
Host: jsonplaceholder.typicode.com       ← ヘッダー
Accept: application/json

HTTP/1.1 200 OK                          ← ステータス行
Content-Type: application/json; charset=utf-8

{"userId": 1, "id": 1, "title": "...", "body": "..."}   ← ボディ
```

ヘッダーとボディの間には空行が 1 行入ります。ブラウザの DevTools や Python の `requests` は，このメッセージを組み立てたり分解したりする作業を代わりに行ってくれます。

2.5 ステータスコード

ステータスコードは 100 番台から 500 番台までの 5 分類で理解すると分かりやすくなります。**API では，成功時よりも失敗時の読み解きが重要**です。

分類	範囲	意味	代表例
情報	100 番台	処理中の情報	100 Continue
成功	200 番台	要求が成功した	200 OK, 201 Created
リダイレクト	300 番台	別の場所へ誘導	301 Moved Permanently
クライアントエラー	400 番台	要求側の誤り	400, 401, 403, 404
サーバーエラー	500 番台	サーバー側の問題	500 Internal Server Error

2.6　API でよく見るステータスコード

5 分類を覚えたら，次は API 実習で頻繁に見る番号を押さえます。番号だけを暗記するのではなく，**次に何を確認するか**とセットで考えることが重要です。

コード	意味	次に確認すること
200	OK。取得や更新が成功した	レスポンス JSON の中身が期待どおりか
201	Created。新しいリソースが作成された	作成された ID や Location ヘッダーがあるか
204	No Content。成功したが本文はない	JSON を読もうとしていないか
400	Bad Request。リクエスト形式が不正	必須パラメータ，型，単位，日付形式
401	Unauthorized。認証されていない	API キーやトークンの有無・期限
403	Forbidden。認可されていない	権限，利用プラン，IP 制限，利用規約
404	Not Found。対象が見つからない	URL，ID，エンドポイントの綴り
429	Too Many Requests。呼び出し過多	レート制限の残り回数とリセット時刻
500	Internal Server Error。サーバー側の問題	時間をおいて再試行し，必要なら提供元の障害情報
503	Service Unavailable。一時的に利用できない	メンテナンス情報，`Retry-After` ヘッダー

章末練習：解答例つき

1. 商品データを読み取るだけの場合，どの HTTP メソッドを使うのが自然か。
   解答例：`GET` を使う。読み取り専用の操作として扱えるためである。
2. 認証トークンやデータ形式の指定は，リクエストのどの要素に入ることが多いか。
   解答例：ヘッダーに入ることが多い。例として `Authorization` や `Content-Type` がある。
3. 存在しない URL にアクセスして `404` が返った。この原因をどう推測するか。
   解答例：エンドポイント，リソース ID，URL の綴りが間違っている可能性をまず確認する。

第 3 章

JSON の読み方

3.1　JSON の構造

JSON（JavaScript Object Notation）は，API でもっともよく使われるデータ形式です。基本は次の 4 つだけです。

- **オブジェクト**：`{ }` で囲まれた「キーと値」の集まり
- **配列**：`[ ]` で囲まれた値の並び
- **キー**：項目の名前（文字列）
- **値**：文字列・数値・真偽値（`true`/`false`）・配列・オブジェクト・`null`

次の例を見てみましょう。`title` というキーの値が `"sample title"` です。

```
{
  "userId": 1,
  "id": 1,
  "title": "sample title",
  "body": "sample body"
}
```

ここがポイント

「キーを指定して値を取り出す」── これが JSON 処理の基本動作です。第 III 部では，Python でこの操作を行います（`data["title"]` のように書きます）。

3.2　入れ子と配列を読む

実際の API レスポンスは，単純な 1 階層だけとは限りません。オブジェクトの中にオブジェクトが入り，さらに配列が入ることがあります。

```
{
  "orderId": 123,
```

```
  "customer": {
    "name": "Yamada",
    "rank": "gold"
  },
  "items": [
    {"name": "coffee", "price": 450},
    {"name": "sandwich", "price": 680}
  ]
}
```

この例では，顧客名は `customer` の中の `name` にあります。商品は `items` という配列に並んでおり，先頭の商品名は `items[0]["name"]` のように考えます。JSON を読むときは，次の順序で目を動かすと迷いにくくなります。

1. 最外側が `{ }` か `[ ]` かを確認する。
2. ほしい値のキー名を探す。
3. 途中に配列があれば，何番目の要素を見るのかを確認する。
4. 値が存在しない場合や `null` の場合を考える。

3.3　JSON と Python の対応

Python で `response.json()` を実行すると，JSON は Python の型に変換されます。表記は少し変わりますが，意味は同じです。

JSON	Python	例
オブジェクト	辞書（`dict`）	`{"title": "A"}` は `data["title"]` で取り出す
配列	リスト（`list`）	`[10, 20, 30]` は `data[0]` で先頭を取り出す
文字列	`str`	`"Nagoya"`
数値	`int` / `float`	`123`，`21.5`
真偽値	`bool`	JSON の `true` は Python の `True`
`null`	`None`	値がないことを表す

存在しないキーに注意

`data["title"]` は，`title` が存在しないとエラーになります。外部 API では項目が欠けることがあるため，実務では `data.get("title")` のように値がない場合も扱える書き方を検討します。

演習 2：JSON を読んでみよう

巻末の「演習 2」（A）の JSON を読み，キーと値の対応を確認しましょう。入れ子の例では，顧客ランク（`gold`）と 2 番目の商品の価格（`680`）にたどり着く経路も言葉で説明してみましょう。

章末練習：解答例つき

1. 次の JSON で `title` の値は何か：`{"title": "sample title"}`。
   解答例：`sample title` である。
2. `[ ]` と `{ }` はそれぞれ何を表すか。
   解答例：`[ ]` は配列，`{ }` はキーと値の組を持つオブジェクトを表す。
3. Python で辞書 `data` から `title` を取り出す式を書きなさい。
   解答例：`data["title"]` と書く。
4. 入れ子の例で，2 番目の商品の価格を取り出す式を書きなさい。
   解答例：`data["items"][1]["price"]` と書く。添字は 0 から数えるので 2 番目は `1` である。

第 II 部

体験編：ブラウザで API を試す

最初の実習では，プログラミングを使わずに API を呼びます。ここで「**API は特別なものではなく，URL にアクセスするとデータが返ってくるものだ**」と実感しましょう。本書ではツールを追加せず，**ブラウザだけ**で体験します。

第 4 章

アドレスバーで API を呼ぶ

4.1　JSONPlaceholder を開いてみる

ブラウザのアドレスバーに，次の URL を入力してアクセスしてみましょう。

```
https://jsonplaceholder.typicode.com/posts/1
```

ブログ記事のような JSON データが表示されます。多くのブラウザは JSON を自動で見やすく整形して表示します（JSON ビューア）。これが「GET リクエストを送り，レスポンスを受け取る」というもっとも単純な API 利用です。

4.2　URL の末尾を変えてみる

末尾の数字を `/posts/2`，`/posts/10` などに変えると，別の投稿が返ってきます。存在しない番号（例：`/posts/9999`）にすると，ステータスコードは `404` になり，レスポンス本体は `{}` のような空の JSON になることがあります。ブラウザ画面だけではステータスコードが分かりにくいので，次章の DevTools で確認します。

演習 1：API にアクセスしてみよう

ブラウザで `/posts/1` にアクセスし，巻末の「演習 1 ワークシート」に URL・データ形式・見つけたキー名とその値を記入しましょう。余裕があれば `/users/1` や `/todos/1` も開き，巻末「課題 A」（B）の表にまとめます。

章末練習：解答例つき

1. ブラウザで `https://jsonplaceholder.typicode.com/posts/1` を開くと，どの HTTP メソッドのリクエストになるか。
   解答例：`GET` リクエストになる。
2. `/posts/1` を `/posts/10` に変えると何が変わるか。
   解答例：取得対象の投稿 ID が変わり，別の投稿データが返る。
3. `/posts/9999` のように存在しない番号を指定した場合，何を確認すべきか。

解答例：画面の本文だけでなく，DevTools の Network タブでステータスコードが 404 か確認する。

第 5 章

開発者ツール（DevTools）で観察する

アドレスバーでは GET しか送れず，ステータスコードも見えません。ブラウザの **開発者ツール（DevTools）**を使うと，リクエストの中身まで観察でき，GET 以外のメソッドも試せます。Windows では `F12` キー（または `Ctrl+Shift+I`），macOS では `Cmd+Option+I` で開きます。

5.1　Network タブで観察する

DevTools の **Network** タブを開いた状態でページを再読み込みすると，送られたリクエストの一覧が見えます。各リクエストをクリックすると，

- ステータスコード（200 など）
- リクエスト／レスポンスのヘッダー
- レスポンス本体（JSON）

を確認できます。第 2 章で学んだ「リクエスト・レスポンスの構造」を，実物で確かめましょう。

Network タブで見る順番

リクエストを選んだら，まず **Status**，次に **Request URL**，**Request Headers**，**Response Headers**，**Response** / **Preview** の順に見ます。本文だけを見ると成功したように見えても，ステータスコードやヘッダーに重要な情報が出ていることがあります。

5.2　Console で fetch() を使う

DevTools の **Console** タブでは，JavaScript を直接実行できます。`fetch()` を使うと，ブラウザだけで API を呼び出せます（GET の例）。

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data));
```

Console を開くページ

Console の `fetch()` は，いま開いているページから送られます。サイトによってはセキュリティ設定（CORS や CSP）によって他サイトへの通信が拒否されることがあります。練習するときは，先に https://jsonplaceholder.typicode.com/ を開き，そのページの Console で実行してください。

5.2.1 POST・PUT・DELETE も試す

専用ツールがなくても，Console から POST・PUT・DELETE を送れます。JSONPlaceholder は**疑似 API** なので，安全に練習できます（実際にはデータは変わりません）。

```
// 新しい投稿を作成する（POST）
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "私の投稿", body: "本文", userId: 1 })
})
  .then(r => r.json())
  .then(console.log);
```

PUT や DELETE も同じ考え方で `method` を変えて送信します。

```
// 1番の投稿を置き換える（PUT）
fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ id: 1, title: "更新後", body: "本文", userId: 1 })
})
  .then(r => r.json())
  .then(console.log);

// 1番の投稿を削除する（DELETE）。ステータスコードだけ表示する
fetch("https://jsonplaceholder.typicode.com/posts/1", { method: "DELETE" })
  .then(r => console.log(r.status));
```

JSONPlaceholder では作成・更新・削除が「成功したように」返りますが，サーバー上のデータは実際には変更されません。

5.2.2 ブラウザ体験で確認したいこと

この章の目的は，JavaScript を覚えることではありません。Python に進む前に，次の対応関係を自分の目で確認することです。

確認するもの	何を理解するためか
Request URL	エンドポイントとクエリパラメータがどう組み合わさるか
Request Method	GET，POST，PUT，DELETE のどれが送られたか
Status Code	成功か，利用者側の誤りか，サーバー側の問題か
Content-Type	返ってきたデータが JSON なのか HTML なのか
Response Body	必要な値がどのキーに入っているか

観察の問い

1. このリクエストは何を取得（作成）しているか。
2. ステータスコードは何番か。
3. レスポンスの中で，記事タイトルはどの項目か。
4. URL の最後の数字を変えると何が起きるか。

章末練習：解答例つき

1. Network タブで確認できる情報を 3 つ挙げなさい。
   解答例：ステータスコード，リクエストヘッダー，レスポンス本体を確認できる。
2. Console で `fetch("https://jsonplaceholder.typicode.com/posts/1")` を実行すると，何をしているか。
   解答例：ブラウザから投稿 ID 1 の API に GET リクエストを送っている。
3. JSONPlaceholder に POST を送った結果は，本当にサーバー上に保存されるか。
   解答例：保存されない。成功したようなレスポンスは返るが，疑似 API なので実データは変更されない。

第 III 部

実装編：Python で API を呼び出す

ここからは Python で API を呼びます。経営情報システムの文脈では，JavaScript よりも Python のほうが，データ分析や表計算との接続に展開しやすいためです。この段階では REST の厳密な設計論よりも，「**API からデータを取ってきて使う**」体験を優先します。

実行例について

以下に掲載する実行結果は，2026 年 10 月 9 日（JST）にこの教材のスクリプトを実行して得た例です。外部 API の値，GitHub のスター数，天気予報は実行時点によって変わることがあります。

第 6 章

最初のリクエスト

6.1　GET して，ステータスと JSON を見る

最初のスクリプトはとても短いものです。次のコマンドで実行します。

```
uv run scripts/01_first_request.py
```

scripts/01_first_request.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""最初のリクエスト：JSONPlaceholder から投稿を1件取得する。

実行方法:
    uv run scripts/01_first_request.py
"""
import requests

# エンドポイント（API の URL）。末尾の 1 が「投稿番号」を表す。
url = "https://jsonplaceholder.typicode.com/posts/1"

# GET リクエストを送る。timeout で応答待ちの上限（秒）を決めておく。
response = requests.get(url, timeout=10)

# ステータスコード（200 なら成功）を確認する。
print("ステータスコード:", response.status_code)

# レスポンス本体を JSON として取り出し、Python の辞書として表示する。
print(response.json())
```

実行すると，ステータスコード 200 と，取得した JSON が表示されます。

```
ステータスコード: 200
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati
    excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae
    consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum
    rerum est autem sunt rem eveniet architecto'}
```

6.2　API 呼び出しの基本形

この後に扱う API が変わっても，Python 側の基本形は大きく変わりません。まずは次の 6 段階で考えます。

1. エンドポイント URL を決める。
2. 必要なパラメータを辞書で用意する。
3. `requests.get()` や `requests.post()` でリクエストを送る。
4. `status_code` または `raise_for_status()` で成功・失敗を確認する。
5. `response.json()` で JSON を Python の辞書・リストに変換する。
6. 必要なキーだけを取り出し，表・グラフ・業務判断に使う。

JSON とは限らない

API だと思ってアクセスしても，認証エラー画面やメンテナンス画面の HTML が返ることがあります。その場合，`response.json()` は失敗します。実務ではステータスコードと `Content-Type` ヘッダーを確認してから JSON として読むのが安全です。

```
content_type = response.headers.get("Content-Type", "")
if response.ok and "application/json" in content_type:
    data = response.json()
else:
    print("JSON ではない応答:", response.status_code, content_type)
```

6.3　JSON から必要な値を取り出す

`response.json()` が返すのは Python の辞書（`dict`）です。キーを指定すれば，必要な値だけを取り出せます。

scripts/02_extract_values.py

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
```

```
"""JSON から必要な値だけを取り出す。

実行方法:
    uv run scripts/02_extract_values.py
"""
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url, timeout=10)
data = response.json()  # data は Python の辞書（dict）になる

# 辞書からキーを指定して値を取り出す。
print("タイトル:", data["title"])
print("本文:", data["body"])
```

実行例：

```
タイトル: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
本文: quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
```

6.4　この章の要点

命令	説明
`requests.get(url, timeout=10)`	API に GET リクエストを送る。応答待ちの上限も決める
`response.status_code`	成功・失敗を確認する（200 なら成功）
`response.json()`	JSON を Python の辞書・リストとして扱う
`data["title"]`	辞書からキーを指定して値を取り出す

章末練習：実行結果例つき

1. `scripts/01_first_request.py` を実行するコマンドを書きなさい。
   解答例：`uv run scripts/01_first_request.py` と実行する。
2. 実行結果で成功を示すステータスコードは何番か。
   実行結果例：ステータスコード: 200 と表示される。
3. `scripts/02_extract_values.py` は JSON のどのキーを取り出しているか。
   解答例：`title` と `body` を取り出し，タイトルと本文として表示している。

第 7 章

パラメータとエラー処理

7.1　クエリパラメータで絞り込む

`?key=value` の形で条件を付けるのが「クエリパラメータ」です。`requests` では `params=` に辞書を渡すだけで，URL に自動で組み込まれます。

scripts/03_query_params.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""クエリパラメータ（?key=value）を付けてリクエストする。

「投稿番号 1 に付いたコメント」を取得する例。
実行方法:
    uv run scripts/03_query_params.py
"""
import requests

url = "https://jsonplaceholder.typicode.com/comments"

# params に辞書を渡すと、?postId=1 のように URL へ自動で組み込まれる。
params = {"postId": 1}

response = requests.get(url, params=params, timeout=10)
data = response.json()

print("実際に送られた URL:", response.url)
print("取得した件数:", len(data))

# 先頭の3件だけ、投稿者メールと名前を表示する。
for comment in data[:3]:
    print("-", comment["email"], ":", comment["name"])
```

`response.url` を表示すると，実際に送られた URL を確認できます。

実行例：

```
実際に送られた URL: https://jsonplaceholder.typicode.com/comments?postId=1
取得した件数: 5
- Eliseo@gardner.biz : id labore ex et quam laborum
- Jayne_Kuhic@sydney.com : quo vero reiciendis velit similique earum
- Nikita@garfield.biz : odio adipisci rerum aut animi
```

7.1.1　URL を文字列で連結しない理由

`https://example.com/search?q=rest api&limit=10` のような URL を自分で文字列連結しても，単純な場合は動きます。しかし，日本語や空白，記号を含むと URL エンコードが必要になります。`requests` の `params=` を使うと，必要な変換をライブラリに任せられます。

```
params = {
    "q": "REST API 入門",
    "limit": 10,
}
response = requests.get(url, params=params, timeout=10)
```

プログラムでは「URL 本体」と「検索条件」を分けて書くと，読みやすく，変更もしやすくなります。

7.2　エラー処理

実務では，成功よりも **失敗の扱い** が重要です。次のスクリプトは，存在する投稿（200）と存在しない投稿（404）の両方を安全に処理します。

scripts/04_error_handling.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""エラー処理：成功（200）と失敗（404）の両方を扱う。

実行方法:
    uv run scripts/04_error_handling.py
"""
import requests


def fetch_post(post_id: int):
    """投稿を取得する。失敗したら None を返す。"""
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"
```

```
    try:
        response = requests.get(url, timeout=10)
        # 4xx・5xx のときは例外（HTTPError）を発生させる。
        response.raise_for_status()
    except requests.exceptions.HTTPError:
        print(f"[{post_id}] HTTP エラー: ステータスコード {response.status_code}")
        return None
    except requests.exceptions.RequestException as error:
        # 接続できない・タイムアウトなどの通信エラー。
        print(f"[{post_id}] 通信エラー: {error}")
        return None
    return response.json()


# 存在する投稿（200 OK）
post = fetch_post(1)
print("取得成功:", post["title"] if post else "なし")

# 存在しない投稿（404 Not Found）
missing = fetch_post(9999)
print("取得結果:", missing)
```

実行例：

```
取得成功: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
[9999] HTTP エラー: ステータスコード 404
取得結果: None
```

ここがポイント

- `response.raise_for_status()`：4xx・5xx のとき例外を発生させる。
- `try ... except`：通信エラーや HTTP エラーを受け止める。
- `timeout=10`：応答が来ないときに待ち続けないよう上限を設ける。
- `except` は**具体的な例外から順に**書く。`HTTPError` は `RequestException` の一種なので，順序を逆にすると HTTP エラーも「通信エラー」として扱われてしまう。

7.3　ステータスコードから原因を推測する

エラーが出たら，まずステータスコードを見て原因を絞り込みます（詳しい対応は巻末「演習 3」を参照）。

症状	最初に確認すること
タイムアウトする	ネットワーク接続，URL，プロキシ設定，API 側の障害
400 が返る	パラメータ名，値の型，必須項目，日付や単位の指定
401/403 が返る	API キー，トークン，権限，利用プラン
404 が返る	エンドポイントの綴り，ID の存在，API バージョン
429 が返る	レート制限，短時間の再実行，待機すべき時刻
500/503 が返る	提供元の障害情報，時間をおいた再試行，`Retry-After` ヘッダー
JSON の取り出しで失敗する	レスポンスが HTML や空本文ではないか，`Content-Type`

エラー対応の考え方

エラーは「失敗」ではなく，システムからの情報です。まずステータスコードで大分類し，次に URL・パラメータ・ヘッダー・レスポンス本文を確認します。闇雲に再実行すると，レート制限や重複登録につながることがあります。

章末練習：実行結果例つき

1. `scripts/03_query_params.py` で送られた実際の URL は何か。
   実行結果例：`https://jsonplaceholder.typicode.com/comments?postId=1` である。
2. `postId=1` のコメントは何件取得されたか。
   実行結果例：`取得した件数: 5` と表示される。
3. `scripts/04_error_handling.py` で存在しない投稿を取得したとき，何が表示されるか。
   実行結果例：`[9999] HTTP エラー: ステータスコード 404` と表示され，取得結果は `None` になる。

第 IV 部

応用編：実データ API を業務に活かす

応用編では，実際に意味のあるデータを取得します。ここでようやく，技術が「**外部データを業務判断に使う**」という MIS の主題につながります。後半では，データを「読む」だけでなく「作成・更新・削除する」操作（CRUD）も疑似 API で体験します。

第 8 章

天気データを取得する（Open-Meteo）

8.1　現在の気温・風速を取得する

Open-Meteo は天気予報・過去天気データの API を提供しており，**API キーは不要**です。緯度・経度とパラメータの意味に集中できます。ここでは名古屋市を例にします。なお，Open-Meteo の「現在値」は観測所の実測値そのものではなく，気象モデルにもとづく直近の推定値・予報値として扱います。

scripts/05_openmeteo_current.py

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""Open-Meteo から名古屋市の現在の気温・風速を取得する（APIキー不要）。

実行方法:
    uv run scripts/05_openmeteo_current.py
"""
import requests

url = "https://api.open-meteo.com/v1/forecast"

params = {
    "latitude": 35.1815,        # 名古屋市（浜松市なら 34.7108）
    "longitude": 136.9066,      # 名古屋市（浜松市なら 137.7261）
    "current": "temperature_2m,wind_speed_10m",
    "wind_speed_unit": "ms",    # 風速の単位を m/s にする
    "timezone": "Asia/Tokyo",
}

response = requests.get(url, params=params, timeout=10)
response.raise_for_status()
data = response.json()

# まずは生のデータ（辞書）をそのまま表示してみる。
```

```
current = data["current"]
print(current)

# 必要な値だけを取り出して読みやすく表示する。
print("データ時刻:", current["time"])  # モデルによる推定値の時刻（実測の観測時刻ではない）
print("気温(°C) :", current["temperature_2m"])
print("風速(m/s):", current["wind_speed_10m"])
```

実行例（interval の 900 は，値が 900 秒＝ 15 分間隔で更新されることを表します）：

```
{'time': '2026-10-09T09:15', 'interval': 900, 'temperature_2m': 20.9, '
    wind_speed_10m': 0.5}
データ時刻: 2026-10-09T09:15
気温(°C) : 20.9
風速(m/s): 0.5
```

8.2　パラメータの意味

パラメータ	意味
latitude / longitude	緯度・経度（場所の指定）
current	直近時点で知りたい項目（気温・風速など）。カンマ区切りで複数指定できる
hourly	1 時間ごとの予報項目
timezone	時刻の基準（Asia/Tokyo）。省略すると UTC（協定世界時）で返る
forecast_days	何日分の予報を取得するか（既定は 7 日）
wind_speed_unit	風速の単位。既定は km/h，ms を指定すると m/s

8.3　Open-Meteo レスポンスの読み方

Open-Meteo のレスポンスには，データ本体だけでなく単位やタイムゾーンの情報も含まれます。たとえば current の値を読むときは，対応する current_units も確認すると，表示単位の取り違えを防げます。

キー	読み方
`latitude` / `longitude`	API が扱った地点。丸められることがあるため，指定値と完全一致しない場合がある
`timezone`	返される時刻の基準。日本の授業では `Asia/Tokyo` を明示する
`current`	直近時点の気温や風速など
`current_units`	`current` に入っている値の単位
`hourly`	時刻ごとの配列。時刻，気温，降水確率などの配列は同じ順番で対応する
`hourly_units`	`hourly` に入っている値の単位

時間別データでは，同じ添字の値が同じ時刻に対応します。

```
hourly["time"][0]
hourly["temperature_2m"][0]
hourly["precipitation_probability"][0]
```

この「同じ添字で対応する」構造は，時系列 API でよく出てきます。Python では `zip()` を使うと，複数の配列から同じ添字の値を 1 組ずつ取り出せます（次節の `06_openmeteo_table.py` で使います）。

8.4　時間別予報を表にする

取得したデータを，読みやすい表として出力してみます。

scripts/06_openmeteo_table.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""Open-Meteo の時間別予報を取得し、表として整形して表示する。

実行方法:
    uv run scripts/06_openmeteo_table.py
"""
import requests

url = "https://api.open-meteo.com/v1/forecast"

params = {
    "latitude": 35.1815,
    "longitude": 136.9066,
    "hourly": "temperature_2m,precipitation_probability",
    "timezone": "Asia/Tokyo",
    "forecast_days": 1,
```

```
}

response = requests.get(url, params=params, timeout=10)
response.raise_for_status()
data = response.json()

hourly = data["hourly"]
times = hourly["time"]
temps = hourly["temperature_2m"]
pops = hourly["precipitation_probability"]

# 見出し行（ASCII で桁をそろえる）
print(f"{'time':<18}{'temp(C)':>10}{'rain(%)':>10}")
print("-" * 38)

# 先頭から12時間分を表示する。
for time, temp, pop in list(zip(times, temps, pops))[:12]:
    print(f"{time:<18}{temp:>10}{pop:>10}")
```

実行例：

```
time                 temp(C)   rain(%)
--------------------------------------
2026-10-09T00:00        18.3         0
2026-10-09T01:00        17.8         0
2026-10-09T02:00        17.3         0
2026-10-09T03:00        16.9         0
2026-10-09T04:00        16.7         0
2026-10-09T05:00        16.4         0
2026-10-09T06:00        16.1         0
2026-10-09T07:00        16.7         0
2026-10-09T08:00        18.4         0
2026-10-09T09:00        20.4         0
2026-10-09T10:00        22.4         0
2026-10-09T11:00        24.0         0
```

8.5　グラフにする

さらに，気温の推移を折れ線グラフにできます。次のスクリプトは figures/forecast.png を生成します（日本語表示には matplotlib-fontja を使用）。

scripts/07_openmeteo_plot.py

```
# /// script
# requires-python = ">=3.11"
```

```
# dependencies = ["requests", "matplotlib", "matplotlib-fontja"]
# ///
"""Open-Meteo の時間別気温予報を取得し、折れ線グラフとして保存する。

出力: figures/forecast.png
実行方法:
    uv run scripts/07_openmeteo_plot.py
"""
from pathlib import Path

import requests
import matplotlib
matplotlib.use("Agg")  # 画面のない環境でも画像ファイルとして保存できるようにする
import matplotlib.pyplot as plt
import matplotlib_fontja  # noqa: F401  日本語フォントを自動設定する（import するだけで有効）

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 35.1815,
    "longitude": 136.9066,
    "hourly": "temperature_2m",
    "timezone": "Asia/Tokyo",
    "forecast_days": 1,
}

response = requests.get(url, params=params, timeout=10)
response.raise_for_status()
data = response.json()
times = data["hourly"]["time"]
temps = data["hourly"]["temperature_2m"]

# 時刻ラベルは "HH:MM" だけにして見やすくする。
hours = [t[11:16] for t in times]

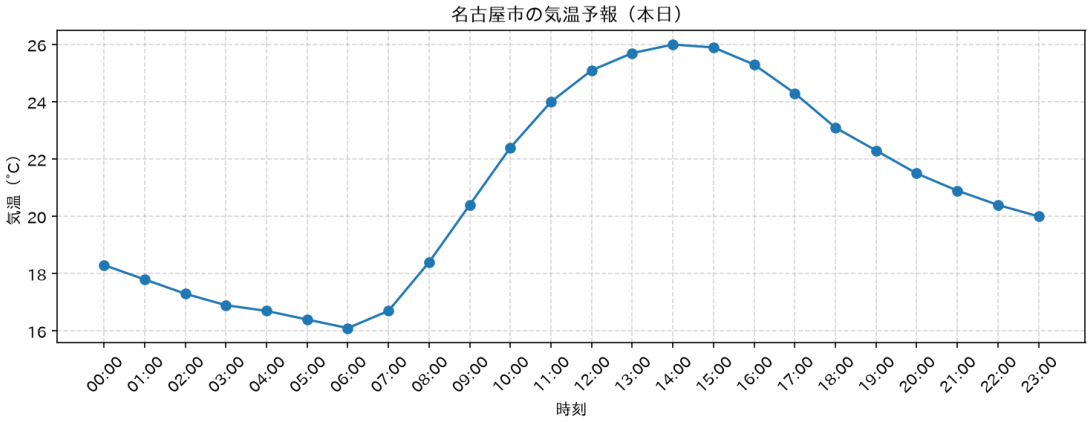
plt.figure(figsize=(10, 4))
plt.plot(hours, temps, marker="o")
plt.title("名古屋市の気温予報（本日）")
plt.xlabel("時刻")
plt.ylabel("気温（°C）")
plt.xticks(rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()

# figures/ ディレクトリへ保存する（このスクリプトの位置を基準にする）。
out_dir = Path(__file__).resolve().parent.parent / "figures"
out_dir.mkdir(exist_ok=True)
out_path = out_dir / "forecast.png"
```

```
plt.savefig(out_path, dpi=150)
print("保存しました:", out_path)
```

実行例：

```
保存しました: /home/katzkawai/kklab-api-tutorial/figures/forecast.png
```



8.6　業務判断に結びつける

取得した気温・風速・天候データは，さまざまな業務判断に使えます。

業務領域	使えるデータ	判断例
小売	気温，降水確率，天候	飲料・弁当・雨具などの発注量，店頭陳列，値引きタイミング
飲食	気温，降水確率，風速	仕込み量，人員配置，テラス席やデリバリー対応
観光	天候，気温，風速	屋外イベントの案内，代替プラン，混雑予測
交通	降水確率，風速，視界関連データ	遅延リスクの周知，運行計画，利用者案内
イベント運営	降水確率，風速，気温	開催判断，備品準備，熱中症・荒天リスク対策

外部データの限界

天気 API の値は意思決定の材料であって，正解そのものではありません。取得時刻，対象地点，単位，更新頻度を記録し，POS データや予約状況など社内データと組み合わせて判断します。

課題 B：天気データを業務に活用する

名古屋市（または任意の都市）の気温・風速を取得し，次の問いに答えなさい。

「気温・風速・天候データは，どのような業務判断に使えるか。小売・観光・交通・イベント運営のいずれかを選んで説明しなさい。」

章末練習：実行結果例つき

1. `scripts/05_openmeteo_current.py` の実行結果で，気温はどのキーから取り出しているか。
   解答例：`current["temperature_2m"]` から取り出している。実行例では `20.9` と表示された。
2. `scripts/06_openmeteo_table.py` の表で `rain(%)` は何を表すか。
   解答例：降水確率（%）を表す。実行例の日は晴天で，どの時刻も `0` だった。降水確率は「雨の量」ではない点にも注意する。
3. 天気データを飲食店で使うなら，どの業務判断に役立つか。
   解答例：気温や降水確率を来客予測，仕込み量，人員配置，テラス席の運用判断に使える。

第 9 章

（発展）GitHub REST API

公開データを返す API として，GitHub REST API も題材になります。認証なしでも利用できますが，**レート制限**（一定時間あたりの呼び出し回数）が厳しめです。目安として，未認証リクエストは送信元 IP アドレスごとに 1 時間 60 回です。認証すると，使えるエンドポイントが増え，レート制限も緩和されます（個人のアクセストークンでは 1 時間 5,000 回が目安）。教室のように多くの学生が同じネットワークから接続すると，IP アドレスが共有されるため上限に早く達することがある点にも注意してください。

9.1　公開リポジトリの情報を取得する

GitHub は，JSON 形式を指定する `Accept` ヘッダーと，API のバージョンを指定する `X-GitHub-Api-Version` ヘッダーを付けて呼び出すことを推奨しています。

scripts/08_github_api.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""（発展）GitHub REST API で公開リポジトリの情報を取得する（認証なし）。

実行方法:
    uv run scripts/08_github_api.py
"""
import requests

# 例として CPython（Python 本体）のリポジトリ情報を取得する。
url = "https://api.github.com/repos/python/cpython"
# GitHub が推奨するヘッダー（JSON 形式と API バージョンを明示する）。
headers = {
    "Accept": "application/vnd.github+json",
    "X-GitHub-Api-Version": "2022-11-28",
}
```

```
response = requests.get(url, headers=headers, timeout=10)
response.raise_for_status()
data = response.json()

print("リポジトリ :", data["full_name"])
print("説明       :", data["description"])
print("スター数   :", data["stargazers_count"])
print("主要言語   :", data["language"])

# レート制限（1時間あたりの呼び出し可能回数）はヘッダーで分かる。
print("レート制限の上限:", response.headers.get("X-RateLimit-Limit"))
print("レート制限の残り:", response.headers.get("X-RateLimit-Remaining"))
```

実行例：

```
リポジトリ : python/cpython
説明       : The Python programming language
スター数   : 77566
主要言語   : Python
レート制限の上限: 60
レート制限の残り: 56
```

レスポンスヘッダーの `X-RateLimit-Limit` で上限，`X-RateLimit-Remaining` で残り呼び出し可能回数が分かります。

9.2 ヘッダーもデータである

API 利用では，レスポンス本文だけでなくヘッダーも重要です。GitHub API の例では，本文にリポジトリ名やスター数が入り，ヘッダーにレート制限情報が入ります。

ヘッダー	使い道
`Content-Type`	返ってきた本文が JSON かどうかを確認する
`X-RateLimit-Limit`	制限期間内に呼び出せる最大回数を確認する
`X-RateLimit-Remaining`	残り呼び出し回数を確認する
`X-RateLimit-Reset`	制限がリセットされる時刻を確認する（UNIX 時間＝ 1970 年 1 月 1 日からの秒数）
`Retry-After`	何秒待ってから再試行すべきかを確認する（429 や 503 のときに付くことがある）

業務システムで API を定期実行する場合は，本文の値だけでなく，こうしたヘッダー情報もログに残すと障害時の原因調査がしやすくなります。

章末練習：実行結果例つき

1. `scripts/08_github_api.py` はどのリポジトリを取得しているか。
   実行結果例：`リポジトリ : python/cpython` と表示される。
2. レート制限の残り回数は，レスポンスのどこから確認しているか。
   解答例：レスポンスヘッダー `X-RateLimit-Remaining` から確認している。
3. GitHub API を認証なしで使うときの注意点を 1 つ挙げなさい。
   解答例：未認証ではレート制限が厳しいため，短時間に何度も実行しない。

第 10 章

CRUD を体験する（疑似 API）

GET だけでなく，**POST（作成）・PUT（置換）・DELETE（削除）** の考え方も体験しておきましょう。実在のサービスのデータを書き換えるのは危険なので，ここでは疑似 API（JSONPlaceholder）に対して安全に練習します。

10.1　CRUD と HTTP メソッドの対応

データ操作の基本 4 種類は，頭文字をとって **CRUD** と呼ばれます。REST API では，それぞれが HTTP メソッドに対応します。

操作	メソッド	URL の例	成功時の代表的なコード
Create（作成）	POST	`/posts`	201 Created
Read（読み取り）	GET	`/posts/1`	200 OK
Update（更新）	PUT / PATCH	`/posts/1`	200 OK（本文なしなら 204）
Delete（削除）	DELETE	`/posts/1`	200 OK または 204 No Content

POST は「新しく作る」ので，まだ ID のない一覧の URL（`/posts`）に送ります。新しい ID はサーバー側が決めて，レスポンスで返します。

10.2　Python で CRUD を送る

`requests` では，`json=` に辞書を渡すと JSON に変換され，`Content-Type: application/json` ヘッダーも自動で付きます。

scripts/09_crud_demo.py

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""疑似 API（JSONPlaceholder）で POST / PUT / PATCH / DELETE を安全に体験する。
```

```
JSONPlaceholder は実際にはデータを変更しないため、練習に安全。
実行方法:
    uv run scripts/09_crud_demo.py
"""
import requests

base = "https://jsonplaceholder.typicode.com/posts"

# --- POST：新しいリソースを作成する（201 Created が返る）---
new_post = {"title": "私の投稿", "body": "本文のサンプルです。", "userId": 1}
created = requests.post(base, json=new_post, timeout=10)
print("[POST]   ステータス:", created.status_code)
print("         作成結果:", created.json())

# --- PUT：既存リソースを置き換える（200 OK が返る）---
updated_post = {"id": 1, "title": "更新後タイトル", "body": "更新後の本文", "userId":
    1}
updated = requests.put(f"{base}/1", json=updated_post, timeout=10)
print("[PUT]    ステータス:", updated.status_code)
print("         更新結果:", updated.json())

# --- PATCH：既存リソースの一部だけを更新する（200 OK が返る）---
patched = requests.patch(f"{base}/1", json={"title": "タイトルだけ変更"}, timeout=10)
print("[PATCH]  ステータス:", patched.status_code)
print("         更新後タイトル:", patched.json()["title"])

# --- DELETE：リソースを削除する（200 OK が返る）---
deleted = requests.delete(f"{base}/1", timeout=10)
print("[DELETE] ステータス:", deleted.status_code)
```

実行例：

```
[POST]   ステータス: 201
         作成結果: {'title': '私の投稿', 'body': '本文のサンプルです。', 'userId': 1, 'id
    ': 101}
[PUT]    ステータス: 200
         更新結果: {'id': 1, 'title': '更新後タイトル', 'body': '更新後の本文', 'userId
    ': 1}
[PATCH]  ステータス: 200
         更新後タイトル: タイトルだけ変更
[DELETE] ステータス: 200
```

POST の結果に `'id': 101` が付いているのは，サーバーが新しい ID を割り当てたことを示します（JSONPlaceholder には投稿が 100 件あるため，次の番号 101 が返ります）。PATCH では `title` だけを送りましたが，レスポンスには `body` など他の項目も残っています。PUT で一部の項目だけを送ると，送らなかった項目が消える（空になる）API もあるため，部分的な変更には PATCH を

使うのが基本です。

10.3　作成・更新・削除で確認すること

GET と違い，POST・PUT・DELETE は業務データを変える可能性があります。教材では疑似 API を使いますが，実務で同じ操作を行う場合は次の点を確認します。

確認項目	内容
対象環境	本番環境か，テスト環境か。誤って本番データを変更しないか
対象リソース	どの ID，どの注文，どの顧客データを変更するか
重複実行	同じ POST を複数回送ったとき，重複作成されないか
戻し方	失敗したとき，データを元に戻す方法があるか
監査ログ	誰が，いつ，何を変更したか記録されるか

注意：本番データを変更しない

POST・PUT・DELETE は，実際にデータを「作成・変更・削除」する操作です。学習段階では，必ず疑似 API やテスト環境で練習してください。

章末練習：実行結果例つき

1. `scripts/09_crud_demo.py` で POST の成功を示すステータスコードは何番か。
   実行結果例：`[POST] ステータス: 201` と表示される。
2. PUT と PATCH の違いを簡単に説明しなさい。
   解答例：PUT はリソース全体の置換，PATCH は一部項目の更新に使う。実行例でも PATCH で `title` だけを送り，他の項目は残った。
3. 学習段階で本番 API に DELETE を送らない理由を説明しなさい。
   解答例：実データが削除され，業務や利用者に影響する可能性があるため，疑似 API やテスト環境で練習する。

第 V 部

運用編：安全な利用と業務システムへの展開

第 11 章

API 利用上の注意

11.1　認証・API キー

本書が最初に認証付き API を使わなかったのは，**認証設定でつまずくと，REST API の本質に入る前に疲れてしまう**からです。実務では多くの API が API キーやトークンによる認証を求めます。鍵は**秘密情報**であり，ソースコードに直接書き込んだり，公開リポジトリに含めたりしてはいけません。

```
# 悪い例：ソースコードに秘密情報を書く
api_key = "YOUR_SECRET_API_KEY"

# よい例：環境変数から読む
import os
api_key = os.environ["SERVICE_API_KEY"]

# 多くの API では，鍵をヘッダーに付けて送る（形式は API ごとに確認する）
headers = {"Authorization": f"Bearer {api_key}"}
response = requests.get(url, headers=headers, timeout=10)
```

環境変数は，実行前にターミナルで設定します。

```
# macOS / Linux
export SERVICE_API_KEY="取得したキー"

# Windows（PowerShell）
$env:SERVICE_API_KEY="取得したキー"
```

API キーを URL のクエリパラメータに付ける API もありますが，URL はブラウザ履歴やサーバーのログに残りやすいため，選べる場合はヘッダーで送るほうが安全です。
授業やレポートで認証付き API を扱う場合も，提出物に API キーを含めないようにします。画面キャプチャや実行ログにもキーが出ることがあるため，提出前に確認してください。

11.2　レート制限・利用規約

多くの API には**レート制限**（一定時間あたりの呼び出し回数の上限）があります。また各 API には**利用規約**があり，用途・再配布・商用利用の可否などが定められています。組織で使う前に必ず確認します。レート制限に達した場合は，すぐに再試行を繰り返さず，レスポンスヘッダーの残り回数やリセット時刻を見て待機します。

11.3　個人情報・データ品質

取得したデータに個人情報が含まれる場合は，保管・利用に法令や学内規程が関わります。また外部データは**常に正しいとは限りません**。欠損や遅延を前提に，データ品質を確かめる姿勢が必要です。

11.4　ページング・キャッシュ・リトライ

API を実務で使うと，1 回のリクエストでは全件を取得できないことがあります。検索結果や一覧 API では，**ページング**によってデータを分割して取得します。また，同じデータを何度も取りに行くと遅くなり，レート制限にも近づくため，更新頻度に応じて**キャッシュ**を使うことがあります。

運用上の工夫	内容
ページング	`page`，`per_page`，`offset` などで分割取得する
キャッシュ	同じ結果を短時間に何度も取得せず，一時保存した値を使う
リトライ	一時的な失敗（タイムアウト，5xx）だけ再試行する。400・401・404 などは何度送っても結果が変わらないので再試行しない。429 は待機してから再試行する
バックオフ	連続失敗時は待ち時間を長くし，API 提供元に負荷をかけない
ログ	URL，ステータスコード，実行時刻，処理件数を記録する

リトライの注意

POST のようにデータを作成する操作を安易にリトライすると，重複登録が起きることがあります。読み取り中心の GET と，作成・更新系の POST/PUT/DELETE は分けて設計します。

11.4.1　ページングとリトライを組み合わせる

次のスクリプトは，JSONPlaceholder の投稿 100 件を 30 件ずつ分割して取得します。`_page`（ページ番号）と `_limit`（1 ページの件数）は JSONPlaceholder 独自のパラメータ名で，API によって `page`・`per_page`・`offset` などと名前が異なります。また，`urllib3` の `Retry` を使い，429 や 5xx のときだけ，待ち時間を伸ばしながら GET を自動で再試行するように設定しています（`Retry-After` ヘッダーがあればその秒数待ちます）。

scripts/10_pagination_retry.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["requests"]
# ///
"""ページングとリトライ：一覧 API を少しずつ取得し、一時的な失敗は自動で再試行する。

JSONPlaceholder の一覧は _page（ページ番号）と _limit（1ページの件数）で分割できる。
実行方法:
    uv run scripts/10_pagination_retry.py
"""
import time

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

# 一時的な失敗（429・5xx）のときだけ、待ち時間を伸ばしながら最大3回まで再試行する。
# 再試行は GET に限定し、POST などの重複作成を防ぐ。
retry = Retry(
    total=3,
    backoff_factor=1,
    status_forcelist=[429, 500, 502, 503, 504],
    allowed_methods=["GET"],
)
session = requests.Session()
session.mount("https://", HTTPAdapter(max_retries=retry))

url = "https://jsonplaceholder.typicode.com/posts"
page = 1
all_posts = []

while True:
    params = {"_page": page, "_limit": 30}
    response = session.get(url, params=params, timeout=10)
    response.raise_for_status()
    posts = response.json()
    if not posts:  # 空のリストが返ったら最後のページを過ぎている
        break

    all_posts.extend(posts)
    total = response.headers.get("X-Total-Count")  # 全件数はヘッダーに入っている
    print(f"{page}ページ目: {len(posts)}件取得（累計 {len(all_posts)} / 全 {total} 件）
    ")
    page += 1
    time.sleep(0.5)  # 提供元に負荷をかけないよう、少し間隔をあける

print("取得完了:", len(all_posts), "件")
```

実行例：

```
1ページ目: 30件取得（累計 30 / 全 100 件）
2ページ目: 30件取得（累計 60 / 全 100 件）
3ページ目: 30件取得（累計 90 / 全 100 件）
4ページ目: 10件取得（累計 100 / 全 100 件）
取得完了: 100 件
```

全件数（`X-Total-Count`）がヘッダーに入っている点にも注目してください。GitHub API のように，次のページの URL を `Link` ヘッダーで知らせる API もあります。

章末練習：解答例つき

1. API キーをソースコードに直接書いてはいけない理由を説明しなさい。
   解答例：公開リポジトリや共有資料から漏えいし，不正利用や課金被害につながるためである。
2. レート制限に達した場合，すぐに再試行を繰り返してよいか。
   解答例：よくない。残り回数やリセット時刻を確認し，待機してから再試行する。
3. 外部データを業務判断に使う前に確認すべきことを 2 つ挙げなさい。
   解答例：データの取得時刻，欠損，更新頻度，利用規約，個人情報の有無などを確認する。
4. `404 Not Found` が返ったリクエストを自動で何度も再試行すべきでない理由を説明しなさい。
   解答例：URL や ID の誤りが原因であり，同じリクエストを送り直しても結果は変わらないためである。提供元への無駄な負荷にもなる。

第 12 章

業務システムでの活用（MIS の視点）

REST API は，企業の業務システム同士をつなぐ「のり」の役割を果たします。

分野	API 連携の例
CRM	顧客情報を他システムと同期する
ERP	在庫・会計データを部門間で連携する
BI	外部データを取り込み，分析・可視化する
RPA	定型業務の中で API を呼び自動化する
SaaS 連携	クラウドサービス間でデータを受け渡す

「API を呼べる」だけでなく，**どの業務を，どのデータで，どう改善できるか**を考えられることが，経営情報システムを学ぶ意義です。

12.1　API 連携を設計する手順

業務システムで API を使うときは，先にコードを書くのではなく，業務上の目的から逆算します。次の順序で整理すると，技術だけの提案になりにくくなります。

1. **業務課題を決める**：例として，発注量のばらつき，人員配置の過不足，手入力の重複など。
2. **必要な意思決定を決める**：何を判断するためにデータが必要なのかを明確にする。
3. **必要なデータを決める**：外部 API のデータと社内データを分けて考える。
4. **更新頻度を決める**：リアルタイムで必要か，1 日 1 回でよいかを判断する。
5. **連携先を決める**：POS，予約管理，在庫管理，BI ツールなど，どこに渡すかを決める。
6. **リスクを確認する**：認証，利用規約，個人情報，欠損，障害時の代替手順を確認する。

MIS の視点

API 連携の価値は「データを取れた」ことではなく，判断の質，処理時間，人的ミス，顧客体験のどれをどの程度改善できるかで評価します。

章末練習：解答例つき

1. CRM と API 連携の例を 1 つ挙げなさい。
   解答例：問い合わせ管理システムと顧客データベースを API で同期し，顧客対応履歴を共有する。
2.「API を呼べる」だけでは不十分な理由を説明しなさい。
   解答例：業務上は，取得したデータでどの判断を改善し，どの効果を出すかまで設計する必要があるためである。
3. 天気 API と POS を組み合わせる業務改善案を 1 つ書きなさい。
   解答例：気温・降水確率と売上履歴を組み合わせ，飲料や弁当の発注量を調整する。

第 VI 部

発展編：コードを整理し，仕様を読む

発展編では，ここまでの「動くコード」を実務で保守しやすい形に整理する方法と，大規模な API 連携で使われる仕様書（OpenAPI）の読み方を紹介します。授業時間が限られる場合は読み飛ばしてもかまいません。

第 13 章

Python ライブラリで API 利用を楽にする

ここまでの例では，`requests.get()` に URL やパラメータを直接渡して API を呼びました。小さな実習ではこれで十分ですが，実務では同じ API を何度も呼ぶため，**認証，ヘッダー，エラー処理，ベース URL，レスポンス処理**を毎回書くとミスが増えます。そこで，API アクセスを抽象化するライブラリや SDK を使います。

13.1　代表的なライブラリの使い分け

種類	代表例	使いどころ
HTTP クライアント	`requests`	本書の標準。GET/POST，`params=`，`json=`，`raise_for_status()` を覚えるだけで多くの API を扱える。
高機能 HTTP クライアント	`httpx`	`requests` に近い書き方で，同期・非同期の両方に対応する。`Client` を使うとベース URL やヘッダーをまとめられる。
非同期クライアント	`aiohttp`	多数の URL を同時に取得する場合に使う。初学者はまず同期処理に慣れてからでよい。
サービス別 SDK	GitHub，Google，Stripe などの SDK	エンドポイント名やページング処理をライブラリ側が隠してくれる。認証付き API や業務システム連携で便利。
生成クライアント	OpenAPI Generator など	OpenAPI 仕様書から Python クライアントを自動生成する。大規模 API やチーム開発向け。

選び方

最初は `requests` で URL，パラメータ，JSON，ステータスコードを理解します。同じ API を何度も呼ぶようになったら `requests.Session` や `httpx.Client` で共通設定をまとめます。公式 SDK がある API では，低レベルな HTTP 呼び出しより SDK を使うほうが保守しやすい場合があります。

13.2 requests.Session で共通設定をまとめる

Session を使うと，ヘッダーや接続を使い回せます。同じサーバーへの接続が再利用されるため，連続して呼び出すときは速くなります。API キーが必要なサービスでは，認証ヘッダーもここにまとめます（10_pagination_retry.py ではリトライ設定もまとめました）。

```
import requests

session = requests.Session()
session.headers.update({"Accept": "application/json"})

params = {
    "latitude": 35.1815,
    "longitude": 136.9066,
    "current": "temperature_2m",
    "timezone": "Asia/Tokyo",
}
response = session.get(
    "https://api.open-meteo.com/v1/forecast",
    params=params,
    timeout=10,
)
response.raise_for_status()
current = response.json()["current"]
print(current["temperature_2m"])
```

13.3 httpx.Client でベース URL を隠す

httpx では Client に base_url を設定できます。呼び出し側はパスだけを書けばよくなります。なお，httpx は requests と違い，既定ではリダイレクト（300 番台）を自動では追いかけません（必要なら follow_redirects=True を指定します）。

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
import httpx

params = {
    "latitude": 35.1815,
    "longitude": 136.9066,
    "current": "temperature_2m",
    "timezone": "Asia/Tokyo",
```

```
}
with httpx.Client(base_url="https://api.open-meteo.com", timeout=10.0) as client:
    response = client.get("/v1/forecast", params=params)
    response.raise_for_status()
    data = response.json()

print(data["current"]["temperature_2m"])
```

13.4　小さな API クライアントを書く

ライブラリだけでなく，自分のコードでも API 呼び出しを関数にまとめると再利用しやすくなります。呼び出す側は URL の組み立てを意識せず，`client.current_temperature()` のような業務寄りの名前で使えます。

```
import requests

class OpenMeteoClient:
    def __init__(self):
        self.session = requests.Session()
        self.base_url = "https://api.open-meteo.com/v1/forecast"

    def current_temperature(self, latitude, longitude):
        params = {
            "latitude": latitude,
            "longitude": longitude,
            "current": "temperature_2m",
            "timezone": "Asia/Tokyo",
        }
        response = self.session.get(self.base_url, params=params, timeout=10)
        response.raise_for_status()
        return response.json()["current"]["temperature_2m"]

client = OpenMeteoClient()
print(client.current_temperature(35.1815, 136.9066))
```

抽象化しすぎない

1 回だけ使う小さな実習では，関数やクラスを増やしすぎると処理の流れが見えにくくなります。同じ API 呼び出しが複数箇所に出てきたとき，認証やエラー処理を共通化したいときに，はじめて小さなクライアントへまとめるくらいで十分です。

13.5 SDK と OpenAPI 生成クライアント

SDK は，`/repos/python/cpython` のような URL ではなく，`get_repo("python/cpython")` のようなメソッドで API を呼べるようにするライブラリです。ページング，認証，リトライ，型変換を隠してくれる一方，内部でどの HTTP リクエストが送られるかは見えにくくなります。学習段階では，まず HTTP の基本を理解してから SDK を使うと仕組みを追いやすくなります。

OpenAPI 仕様書が公開されている API なら，OpenAPI Generator などで Python クライアントを生成できます。

```
openapi-generator-cli generate -g python -i openapi.yaml -o generated-client/
```

生成クライアントは便利ですが，生成されたコードの量が多くなるため，小さな実習では手書きの関数や `requests.Session` のほうが理解しやすい場合があります。

章末練習：解答例つき

1. 本書の標準として使っている HTTP クライアントライブラリは何か。
   解答例：`requests` である。
2. `requests.Session` や `httpx.Client` を使う利点を 1 つ挙げなさい。
   解答例：共通ヘッダー，ベース URL，接続，タイムアウトなどをまとめられ，重複や設定漏れを減らせる。
3. `OpenMeteoClient.current_temperature()` の例では，呼び出し側は何を意識せずに済むか。
   解答例：API の URL，クエリパラメータ，レスポンス JSON の階層を毎回意識せず，気温取得という目的で呼び出せる。

第 14 章

OpenAPI で API 仕様を読む

OpenAPI Specification（旧称 Swagger）は，HTTP API の機能を人間とコンピュータの双方が理解できるように記述する，言語非依存の標準的なインターフェース記述です。仕様書は YAML または JSON で書かれ，`openapi.yaml` や `openapi.json` という名前で公開されることが多いです。これにより，API ドキュメントの作成，クライアントコードの自動生成，テストの自動化などが可能になります。大規模なシステム連携を扱うときの発展トピックとして押さえておきましょう。

14.1　OpenAPI 仕様書に書かれること

OpenAPI 仕様書には，エンドポイント，HTTP メソッド，パラメータ，レスポンス形式，認証方式などが機械可読な形で書かれます。たとえば，投稿を 1 件取得する API は概念的には次のように表せます。

```
openapi: 3.1.0
info:
  title: Sample API
  version: 1.0.0
paths:
  /posts/{id}:
    get:
      summary: 投稿を1件取得する
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: 取得成功
        "404":
          description: 投稿が見つからない
```

このように仕様書を読むと，コードを書く前に「どの URL に何を渡すか」「成功時と失敗時に何が返るか」を確認できます。`{id}` はパスパラメータを表し，`in: path` と `required: true` から「URL のパスに必ず整数の ID を入れる」と読み取れます。Swagger UI などのツールを使うと，仕様書からブラウザで試せる対話的なドキュメントが作られます。

14.2　OpenAPI を使う利点と注意点

観点	内容
ドキュメント	仕様書から人間向けの API ドキュメントを生成できる
コード生成	Python などのクライアントコードやサーバー側の雛形を生成できる
テスト	仕様にないレスポンスやパラメータの不足を検出しやすい
チーム開発	API 提供側と利用側が，同じ仕様書を見ながら作業できる
注意点	仕様書が古いと，生成物も古い。実装と仕様の同期が必要

章末練習：解答例つき

1. OpenAPI Specification は何を記述する仕様か。
   解答例：HTTP API のエンドポイント，メソッド，パラメータ，レスポンス，認証方式などを機械可読に記述する仕様である。
2. OpenAPI 仕様書から自動化できることを 2 つ挙げなさい。
   解答例：API ドキュメント生成，クライアントコード生成，テスト生成などができる。
3. 小さな実習で生成クライアントより手書きコードが向く場合がある理由を説明しなさい。
   解答例：生成コードは量が多く，初学者には HTTP の基本や JSON 処理の流れが見えにくくなる場合があるためである。

付録 A

ワークシート

演習 1：API にアクセスしてみよう

項目	記入欄
アクセスした URL	
HTTP メソッド	
ステータスコード	
返ってきたデータ形式	
見つけたキー名	
そのキーの値	

演習 2：JSON を読んでみよう

次の JSON を見て，問いに答えなさい。

```
{
  "userId": 1,
  "id": 1,
  "title": "sample title",
  "body": "sample body"
}
```

1. `title` の値は何か。
2. `body` にはどのような内容が入っているか。
3. このデータは，業務システムではどのような場面で使えそうか。

演習 3：API 利用の失敗を考えよう

エラー	原因の例	対応方法
400	リクエストの書き方が間違っている	パラメータを確認する
401	認証されていない	API キーやトークンを確認する
403	権限がない	アクセス権限を確認する
404	URL が間違っている	エンドポイントを確認する
429	呼び出し回数が多すぎる	待機してから再試行する
500	サーバー側の問題	時間をおいて再試行する

付録 B

ミニ課題

課題 A：API レスポンス観察

JSONPlaceholder の API を 3 種類呼び出し，それぞれの取得データと使い道を表にまとめなさい。

API	取得できるデータ	業務での類似例
`/posts`	投稿データ	お知らせ・記事・商品説明
`/users`	ユーザー情報	顧客情報・社員情報
`/todos`	ToDo 情報	タスク管理・業務進捗

課題 B：天気データを業務に活用する

第 8 章の演習を発展させ，選んだ業界（小売・観光・交通・イベント運営など）について，天気データをどの業務判断に，どう使えるかを具体的に説明しなさい。

課題 C：API 連携の業務改善提案（最終課題）

最終課題として，次の形式で「API 連携による業務改善提案」をまとめなさい。技術レポートではなく，**業務改善の提案**として書くことがポイントです。

項目	内容（記入例）
対象業務	例：飲食店の来客予測
利用する API	例：天気 API・地図 API・SNS API
取得するデータ	気温・降水量・位置情報など
連携先システム	POS・予約管理・在庫管理など
期待される効果	発注精度向上・人員配置最適化など
注意点	API 制限・個人情報・データ品質など

付録 C

評価ルーブリック

評価観点	A	B	C
REST API の理解	URL・メソッド・JSON・ステータスコードを正しく説明できる	主要な用語は説明できる	用語理解が不十分
API 実行	自力で API を呼び，結果を確認できる	指示があれば実行できる	実行手順が不安定
JSON 処理	必要な値を取り出して加工できる	単純な値は取り出せる	JSON 構造の理解が弱い
エラー対応	ステータスコードから原因を推測できる	よくあるエラーは判断できる	エラー時に対応できない
業務応用	API 連携を業務改善に結びつけて説明できる	一般的な活用例を説明できる	技術説明に留まる

付録 D

API 利用チェックリスト

実行前チェック

新しい API を呼び出す前に，次の項目を確認します。

確認	項目	内容
□	**エンドポイント**	URL と API バージョンは正しいか
□	**メソッド**	GET，POST，PUT，PATCH，DELETE のどれを使うか
□	**パラメータ**	必須項目，単位，日付形式，既定値を確認したか
□	**認証**	API キーやトークンが必要か。提出物に秘密情報が出ないか
□	**レスポンス**	必要な値がどのキーに入るか。配列かオブジェクトか
□	**失敗時**	400，401，403，404，429，500 が返ったときの対応を考えたか
□	**利用条件**	レート制限，商用利用，再配布，出典表示を確認したか

レポート・提案書チェック

API 連携を業務改善として説明するときは，技術説明だけで終わらせないようにします。

確認	項目	内容
□	**業務課題**	何に困っている業務を改善するのかが明確か
□	**利用データ**	外部 API のデータと社内データの役割を分けて説明したか
□	**判断への接続**	取得した値が，発注・配置・案内などの判断にどうつながるか
□	**効果**	時間短縮，精度向上，ミス削減，顧客体験などの効果を書いたか
□	**リスク**	API 障害，データ欠損，個人情報，利用規約への対応を書いたか

付録 E

よくあるトラブルと切り分け

症状	よくある原因	対応
`ModuleNotFoundError`	`python` で直接実行し，PEP 723 の依存関係が読み込まれていない	`uv run scripts/ファイル名.py` で実行する
タイムアウト	ネットワーク，プロキシ，API 提供元の一時障害	URL をブラウザで開く，時間をおく，学内ネットワーク設定を確認する
文字化け	レスポンスや端末の文字コード表示の問題	JSON として取得できているか，端末の表示設定を確認する
`KeyError`	想定したキーがレスポンスに存在しない	生の JSON を表示し，キー名と階層を確認する
`JSONDecodeError`	JSON ではなく HTML や空本文が返っている	ステータスコードと `Content-Type` を確認する
429 が返る	短時間に呼び出しすぎた	再実行を止め，リセット時刻まで待つ
`SSLError`	学内プロキシや PC の時刻設定，証明書の問題	PC の日時を確認し，学内ネットワークの案内に従う。証明書検証を無効にしない
401/403 が返る	API キーの未設定・誤り・期限切れ，権限不足	環境変数が設定されているか，キーの権限を確認する
PDF に図が出ない	`figures/forecast.png` がない	`make figure` または `uv run scripts/07_openmeteo_plot.py` を実行する

付録 F

授業計画（シラバス例）

F.1　90 分 × 1 回（1 回完結）

時間	内容
0–15 分	API とは何か，REST API とは何か
15–30 分	HTTP メソッド・URL・JSON・ステータスコード
30–50 分	JSONPlaceholder をブラウザで呼ぶ
50–70 分	Python で GET リクエストを送る
70–85 分	天気 API を使ったミニ演習
85–90 分	振り返り

この場合は POST や認証までは扱わず，GET 中心にするのがよいでしょう。

F.2　90 分 × 3 回

回	テーマ	内容
第 1 回	REST API 入門	API・HTTP・JSON・GET リクエスト・ブラウザでの観察
第 2 回	Python で API 利用	requests・クエリパラメータ・JSON 処理・エラー処理
第 3 回	業務アプリ連携	天気 API や GitHub API でデータ取得・可視化・考察

F.3　90 分 × 5 回（ミニプロジェクト型）

回	テーマ	成果物
第 1 回	API と REST の基礎	API リクエスト観察シート
第 2 回	ブラウザ演習（DevTools）	GET・POST・PUT・DELETE の実験結果
第 3 回	Python による API 利用	JSON 取得プログラム
第 4 回	エラー処理・認証・利用制限	API 利用時の注意点レポート
第 5 回	ミニプロジェクト	外部 API を使った簡単な業務提案

付録 G

初心者向け公開 API 集

公開 API を選ぶときは，最初から認証や課金設定が必要なものを選ばず，**ブラウザで URL を開くだけで結果を確認できる API** から始めるとよいでしょう。次の一覧は，GET リクエスト，クエリパラメータ，JSON/XML の読み取り，業務活用の検討に使いやすい題材です。なお，仕様や利用条件は変わるため，授業やレポートで使う前に必ず公式ドキュメントを確認してください。

API	認証	まず試す URL	向いている演習
JSONPlaceholder	不要	https://jsonplaceholder.typicode.com/posts/1	投稿・コメント・ユーザーの疑似データ。GET/POST/PUT/DELETE を安全に試せる。実データではない点に注意。
Open-Meteo	不要	https://api.open-meteo.com/v1/forecast?latitude=35.1815&longitude=136.9066&current=temperature_2m&timezone=Asia/Tokyo	天気・気温・降水確率。小売，在庫，イベント運営などの業務判断に結びつけやすい。
Frankfurter	不要	https://api.frankfurter.dev/v1/latest?base=USD&symbols=JPY,EUR	欧州中央銀行の参照レートにもとづく為替レート。輸入価格の試算や為替の時系列グラフに向く。
Open Library	読み取りは不要	https://openlibrary.org/search.json?q=rest%20api	書籍検索。検索結果の配列，著者名，出版年，ページング処理を学びやすい。
World Bank Indicators	不要	https://api.worldbank.org/v2/country/JPN/indicator/SP.POP.TOTL?format=json&date=2022	人口・GDP などの国際統計。時系列データを表やグラフにする練習に向く。
PokéAPI	不要	https://pokeapi.co/api/v2/pokemon/pikachu	教育用に使いやすいゲームデータ。深い JSON，画像 URL，配列の探索を練習できる。
GitHub REST API	公開情報は認証なし可	https://api.github.com/repos/python/cpython	公開リポジトリのスター数，主要言語，ライセンスなど。レート制限ヘッダーの確認にも使える。
NASA APOD	`DEMO_KEY` または API キー	https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY	天文画像と説明文。画像 URL，日付パラメータ，API キーの扱いを説明しやすい。
NDL サーチ API	API キー不要。ただし利用目的により申請・許諾が必要	https://ndlsearch.ndl.go.jp/api/opensearch?cnt=10&title=%E5%A4%8F%E7%9B%AE%E6%BC%B1%E7%9F%B3	国立国会図書館の書誌検索。XML/RSS 形式や日本語検索，利用条件の確認を学べる。

API	認証	まず試す URL	向いている演習
e-Stat API	アプリケーション ID が必要	https://api.e-stat.go.jp/rest/3.0/app/json/getStatsData?appId=YOUR_APP_ID&statsDataId=0003410379	日本の政府統計。ユーザー登録，アプリケーション ID，公的統計の引用・出典表示を学ぶ発展課題向け。

選び方の目安

最初は JSONPlaceholder，Frankfurter，PokéAPI のように認証不要でレスポンスが小さい API を選びます。業務改善提案まで進める場合は Open-Meteo，World Bank，e-Stat のように意思決定に使えるデータを選ぶと，技術説明だけでなく業務上の価値まで考えやすくなります。

付録 H

スクリプト一覧と参考リンク

H.1　スクリプト一覧

いずれも `uv run scripts/ファイル名` の形で実行できます。例えば次のとおりです。

```
uv run scripts/01_first_request.py
```

ファイル	内容
`01_first_request.py`	GET リクエストとステータス・JSON 表示
`02_extract_values.py`	JSON から title・body を取り出す
`03_query_params.py`	クエリパラメータで絞り込む
`04_error_handling.py`	例外処理と 404 の扱い
`05_openmeteo_current.py`	名古屋市の現在の気温・風速
`06_openmeteo_table.py`	時間別予報を表で表示
`07_openmeteo_plot.py`	気温予報をグラフ化し PNG 保存
`08_github_api.py`	GitHub 公開リポジトリ情報（発展）
`09_crud_demo.py`	疑似 API で POST/PUT/PATCH/DELETE
`10_pagination_retry.py`	ページング取得と自動リトライ

H.2　参考リンク

- HTTP リクエストメソッド（MDN）：
  https://developer.mozilla.org/ja/docs/Web/HTTP/Reference/Methods
- HTTP レスポンスステータスコード（MDN）：
  https://developer.mozilla.org/ja/docs/Web/HTTP/Reference/Status
- JSONPlaceholder（練習用の疑似 REST API）：
  https://jsonplaceholder.typicode.com/
- Requests（Python HTTP クライアント）：
  https://requests.readthedocs.io/
- HTTPX（同期・非同期 HTTP クライアント）：

https://www.python-httpx.org/

- aiohttp Client（非同期 HTTP クライアント）：
  https://docs.aiohttp.org/en/stable/client.html
- OpenAPI Generator（クライアント生成）：
  https://openapi-generator.tech/docs/generators/python
- Frankfurter（為替レート API）：
  https://frankfurter.dev/
- Open Library APIs（書籍・著者データ）：
  https://openlibrary.org/developers/api
- World Bank Indicators API（国際統計）：
  https://datahelpdesk.worldbank.org/knowledgebase/articles/889392
- PokéAPI（教育用のゲームデータ API）：
  https://pokeapi.co/docs/v2
- NASA Open APIs / APOD：
  https://api.nasa.gov/
- NDL サーチ API（国立国会図書館）：
  https://ndlsearch.ndl.go.jp/help/api/specifications
- e-Stat API（政府統計）：
  https://www.e-stat.go.jp/api
- Open-Meteo（天気 API）：
  https://open-meteo.com/
- GitHub REST API 入門：
  https://docs.github.com/ja/rest/using-the-rest-api/getting-started-with-the-rest-api
- OpenAPI Specification（OpenAPI Initiative）：
  https://spec.openapis.org/oas/latest.html
- RFC 9110 HTTP Semantics（メソッドの安全性・べき等性の定義）：
  https://www.rfc-editor.org/rfc/rfc9110
- uv（Python 実行・パッケージ管理ツール）：
  https://docs.astral.sh/uv/
- PEP 723 – Inline script metadata：
  https://peps.python.org/pep-0723/